

Lab 3-2 知识文档：前后端分离、API Key 安全与 Skills

对应实验： Lab 3-2：前后端分离实践 — 构建智能出行规划器 建议阅读时机： 开始编码前

一、回顾：前端、后端与 API

在进入 Lab 3-2 之前，先回顾三个基础概念：

概念	你可以怎样理解
前端	用户直接看到和操作的部分，例如页面、地图、表单、按钮、列表
后端	在服务器侧处理数据和业务逻辑的部分，例如保存规划、整合天气、调用大模型
API	一个系统向另一个系统提供能力的接口，例如地图 API、天气 API、大模型 API

在 Lab 3-1 中，你已经体验过：

- 前端页面展示地图与交互
- 调用天气 API 获取外部数据
- 调用大模型 API 生成建议

所以 Lab 3-1 的重点是：会申请 API、会调用 API、会把多个 API 串起来。

而 Lab 3-2 要继续往前走一步：不仅要会调用 API，还要把这些能力组织进一个更完整的软件系统里。

二、Lab 3-1 留下的“坑”

回顾你在 Lab 3-1 写的代码，大模型的调用大概长这样（伪代码）：

```
const res = await
fetch('https://api.deepseek.com/v1/chat/completions', {
  method: 'POST',
  headers: {
```

```
'Content-Type': 'application/json',
'Authorization': 'Bearer sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx'
},
body: JSON.stringify({ model: 'deepseek-chat', messages })
});
```

只要在浏览器里按下 **F12**，任何人都能看到这个 **Authorization** 头里的 API Key。Lab 3-1 实验文档里也明确写了：这是教学场景下的权衡，生产环境中应通过后端代理来保护 Key。

这意味着：

- 纯前端直连大模型 API 并不安全
- API Key 暴露在浏览器里，无法真正保护
- 一旦项目变复杂，前端里会塞进越来越多业务逻辑

Lab 3-2 的一个重要目标，就是把这笔技术债补上。

三、为什么要做前后端分离

仅仅“能调用 API”还不够。一个更完整的软件系统通常还需要：

- 保存用户创建的数据
- 管理多个页面或多个状态
- 对请求进行统一处理
- 避免把所有逻辑都堆在前端页面中
- 更合理地保护第三方 API Key

这就引出了本实验的核心：**前后端分离**。

你可以先直观理解这个变化：

Lab 3-1（不安全）：

浏览器 —Key—> LLM API
—Key—> LLM API

Lab 3-2（前后端分离）：

浏览器 —> 你的后端
 ↑
 Key 只在这里

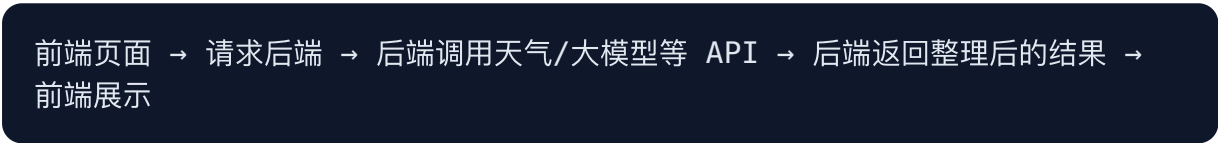
把“敏感凭证 + 业务编排”放到服务端，这是任何真实 Web 应用的标准做法。本质上你给前端做了一个 **BFF（Backend for Frontend）**：前端不再直连关键第三方服务，所有外部调用都先经过你自己的后端。

四、什么是前后端分离

前后端分离不是“把项目拆成两个文件夹”这么简单，它代表了一种职责划分方式：

部分	主要职责
前端	页面展示、地图交互、表单输入、状态切换、结果呈现
后端	数据管理、业务逻辑、天气信息整合、规划保存、大模型调用
第三方 API	提供地图、天气、大模型等外部能力

数据流大致如下：



这带来几个直接好处：

- 前端只关心页面和交互
- 后端统一处理第三方 API 调用
- 业务逻辑不会零散地分布在页面里
- 敏感凭据更容易统一保护
- 后续发布到云服务器时更接近真实项目结构

五、什么是 Skills

在 Trae 中，Skills 可以理解作为一种针对特定任务预先准备好的专用能力包。它们不是普通对话，而是把某一类任务常用的方法、约束和上下文提前组织好了。

你可以把它理解成：

- 普通对话：你需要自己一步步描述需求
- 使用 Skills：Trae 已经带着一套更聚焦的“任务经验”来帮你做事

在 Lab 3-2 中，Skills 的作用主要体现在：

- 帮你先梳理需求和任务拆分
- 帮你设计更好的前端界面
- 帮你整理提交记录

推荐使用的 Skills

Skill	适合场景	建议用途
brainstorming	开发前的需求拆分与方案讨论	在开始编码前，先梳理前后端怎么分工、先做哪些模块
frontend-design	前端页面、布局、交互设计	用于设计规划页面、地点卡片、行程面板、天气展示区域
git-commit	代码完成后的提交整理	帮助生成更规范的提交信息

建议：

- 做前端页面时优先尝试 frontend-design
- 做整体任务拆分时优先尝试 brainstorming

六、为什么默认推荐 Vue 3 + Python + FastAPI + SQLite

Lab 3-2 并不是为了比较各种技术栈，而是为了帮助你完成一次完整的前后端分离实践。因此实验文档中默认推荐：

- 前端：Vue 3
- 后端：Python
- 后端框架：FastAPI
- 数据存储：SQLite

推荐这套技术栈的原因不是“它绝对最好”，而是：

- 便于助教统一讲解
- 便于同学之间互相交流和排查问题
- 便于前后端联调
- 便于后续统一演示如何发布到云服务器
- SQLite 适合课程实验场景，上手快、部署成本低

如果你选择偏离这套推荐栈，就需要自行承担更多适配和排错工作。

七、阅读建议

在正式开始 Lab 3-2 之前，建议你先回答下面几个问题：

1. 为什么 Lab 3-1 的纯前端调用方式不适合继续用于更完整的项目？
2. 在你的系统里，哪些逻辑应当放前端，哪些逻辑应当放后端？

3. 你打算如何保存“规划”这份数据？

4. AI 在这个系统里应该扮演辅助角色，还是主角？为什么？

如果这几个问题你已经有了比较清楚的答案，再去阅读 `lab3_02_v2.md` 的需求部分，会更容易理解本实验真正要做什么。